JSP

PremChandNaidu.G

What is jsp?

Java Server Pages (JSP) is a server-side programming technology that enables the creation of dynamic, platform-independent method for building Web-based applications. JSP have access to the entire family of Java APIs, including the JDBC API to access enterprise databases.

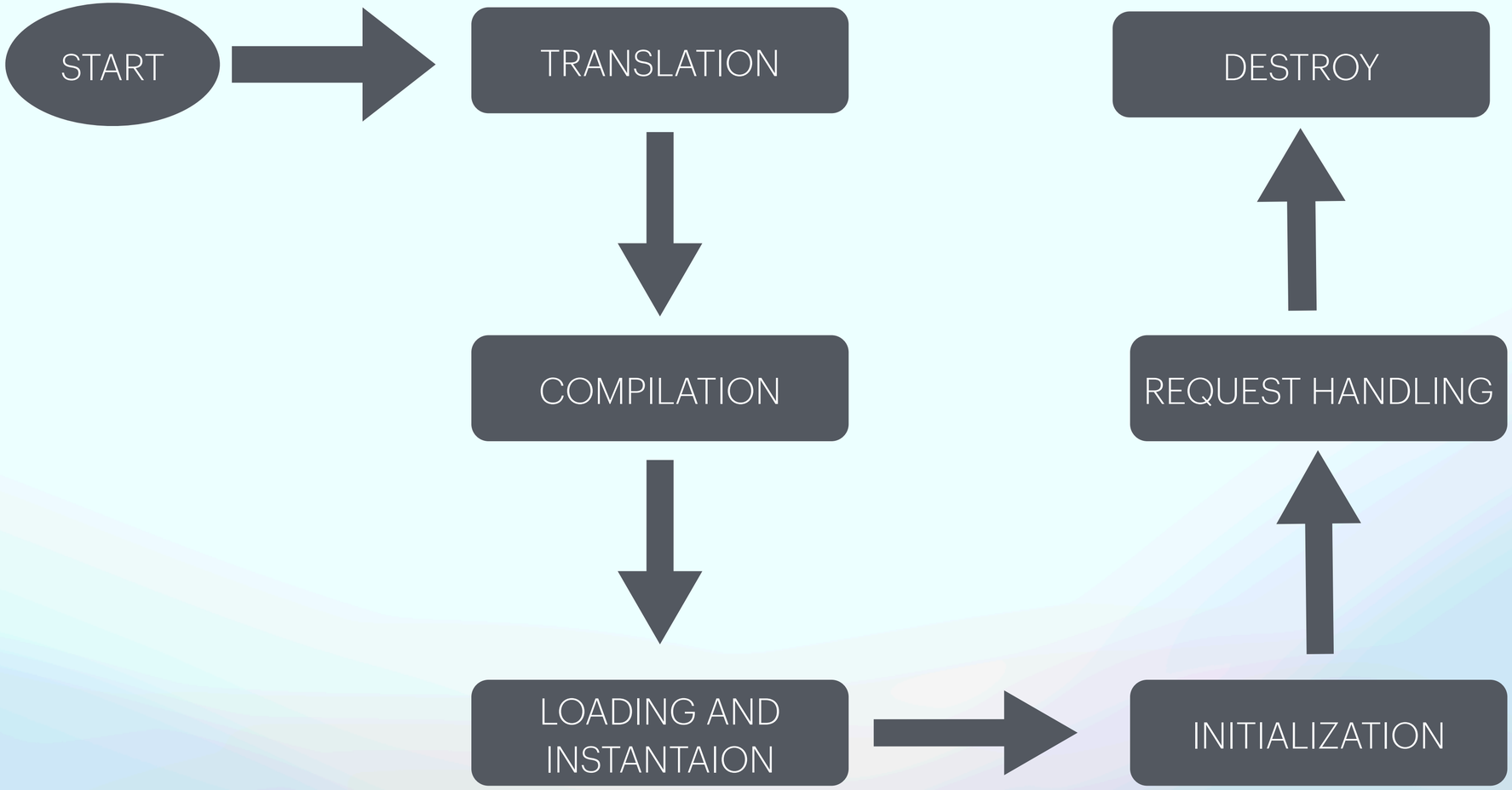
Why to Learn JSP?

- Performance is significantly better because JSP allows embedding Dynamic Elements in HTML Pages itself instead of having separate CGI files.
- JSP are always compiled before they are processed by the server unlike CGI/Perl which requires the server to load an interpreter and the target script each time the page is requested.

- JavaServer Pages are built on top of the Java Servlets API, so like Servlets, JSP also has access to all the powerful Enterprise Java APIs, including **JDBC, JNDI, EJB, JAXP,** etc.
- JSP pages can be used in combination with servlets that handle the business logic, the model supported by Java servlet template engines.

NOTE: BY DEFAULT JSP IS A HTML PAGE, IN THIS WE CAN ALSO USE JAVA CODE. THE IMPORTANCE OF JSP IS TO CREATE DYNAMIC WEB PAGES. ALSO JSP IS ADVANCED VERSION OF SERVLETS.

Life cycle of jsp



- **TRANSLATION:** In this Stage the JSP file is converted into servlet class. As we jsp file is combination html+java, we need to convert JSP file to java file.

- **COMPILATION:** During this stage a jsp file is converted into ".class" file that helps jvm to execute like a java class

- **LOADING AND INSTANTAION:** During this stage when we run tomcat server a new empty jsp container is loaded in server.

- **INITIALIZATION:** At thi stage all the code we written in jsp will be initialized into empty container. This will helps to reduce errors and load for server.

- **REQUEST HANDLING:** all the jsp methods will take request process and respond to the request given by the client.

- **DESTROY:** once our work is done we should destroy all the connections that are opened by server so that no other external resources will not enter into poor databases and servers.

NOTE: For checking the life cycle we can check whether .class and servlet page is created or not in file manager search for jsp, there we can see Catalina/ local host click on the file there we can see the .class file and java file.

ELEMENTS AND DIRECTIVES

JSP SCRIPTING ELEMENTS

In JSP, java code can be written inside the jsp page using the scriptlet tag. Let's see what are the scripting elements first.

JSP Scripting elements

The scripting elements provides the ability to insert java code inside the jsp. There are three types of scripting elements:

- scriptlet tag
- expression tag
- declaration tag

JSP scriptlet tag

A scriptlet tag is used to execute java source code in JSP. Syntax is as follows:

<% java source code %>

Example of JSP scriptlet tag

In this example, we are displaying a welcome message.

```
<html>
<body>
<% out.print("welcome to jsp"); %>
</body>
</html>
```

Example of JSP scriptlet tag that prints the user name

In this example, we have created two files index.html and welcome.jsp. The index.html file gets the username from the user and the welcome.jsp file prints the username with the welcome message.

```
<html>
<body>
<form action="welcome.jsp">
<input type="text" name="uname">
<input type="submit" value="go"><br/>
</form>
</body>
</html>
```

```
<html>
<body>
<%
String name=request.getParameter("uname");
out.print("welcome "+name);
%>
</form>
</body>
</html>
```

JSP expression tag

The code placed within JSP expression tag is written to the output stream of the response. So you need not write out.print() to write data. It is mainly used to print the values of variable or method.

Syntax of JSP expression tag

```
<%=  statement %>
```

Example of JSP expression tag

In this example of jsp expression tag, we are simply displaying a welcome message.

```
<html>
<body>
<%= "welcome to jsp" %>
</body>
</html>
```

Example of JSP expression tag that prints current time

To display the current time, we have used the getTime() method of Calendar class. The getTime() is an instance method of Calendar class, so we have called it after getting the instance of Calendar class by the getInstance() method.

index.jsp

```
<html>
<body>
Current Time: <%= java.util.Calendar.getInstance().getTime() %>
</body>
</html>
```

Example of JSP expression tag that prints the user name

In this example, we are printing the username using the expression tag. The index.html file gets the username and sends the request to the welcome.jsp file, which displays the username.

File: index.jsp

```
<html>
<body>
<form action="welcome.jsp">
<input type="text" name="uname"><br/>
<input type="submit" value="go">
</form>
</body>
</html>
```

File: welcome.jsp

```
<html>
<body>
<%= "Welcome "+request.getParameter("uname") %>
</body>
</html>
```

JSP Declaration Tag

- The JSP declaration tag is used to declare fields and methods.
- The code written inside the jsp declaration tag is placed outside the service() method of auto generated servlet.
- So it doesn't get memory at each request.

Syntax of JSP declaration tag

The syntax of the declaration tag is as follows:

<%! field or method declaration %>

Example of JSP declaration tag that declares field

In this example of JSP declaration tag, we are declaring the field and printing the value of the declared field using the jsp expression tag.

index.jsp

```
<html>
<body>
<%! int data=50; %>
<%= "Value of the variable is:"+data %>
</body>
</html>
```

Example of JSP declaration tag that declares method

In this example of JSP declaration tag, we are defining the method which returns the cube of given number and calling this method from the jsp expression tag. But we can also use jsp scriptlet tag to call the declared method.

index.jsp

```
<html>
<body>
<%!
int cube(int n){
return n*n*n*;
}
%>
<%= "Cube of 3 is:"+cube(3) %>
</body>
</html>
```

JSP directives

The jsp directives are messages that tells the web container how to translate a JSP page into the corresponding servlet.

There are three types of directives:

- page directive
- include directive
- taglib directive

Syntax of JSP Directive

<%@ directive attribute="value" %>

JSP page directive

The page directive defines attributes that apply to an entire JSP page.

Syntax of JSP page directive

<%@ page attribute="value" %>

Attributes of JSP page directive

- import
- contentType
- extends
- info
- buffer
- language
- language
- isELIgnored
- isThreadSafe
- autoFlush
- session
- pageEncoding
- errorPage
- isErrorPage

1)import

The import attribute is used to import class,interface or all the members of a package.It is similar to import keyword in java class or interface.

Example of import attribute

```
<html>
<body>

<%@ page import="java.util.Date" %>
Today is: <%= new Date() %>

</body>
</html>
```

2)contentType

The contentType attribute defines the MIME(Multipurpose Internet Mail Extension) type of the HTTP response.The default value is "text/html;charset=ISO-8859-1".

Example of contentType attribute

```
<html>
<body>

<%@ page contentType=application/msword %>
Today is: <%= new java.util.Date() %>

</body>
</html>
```

3)extends

The extends attribute defines the parent class that will be inherited by the generated servlet.It is rarely used.

4)info

This attribute simply sets the information of the JSP page which is retrieved later by using getServletInfo() method of Servlet interface.

Example of info attribute

```
<html>
<body>
  <%@ page info="composed by Sonoo Jaiswal" %>
Today is: <%= new java.util.Date() %>
  </body>
</html>
```

The web container will create a method getServletInfo() in the resulting servlet.For example:

```
public String getServletInfo() {
  return "composed by Sonoo Jaiswal";
}
```

5)buffer

The buffer attribute sets the buffer size in kilobytes to handle output generated by the JSP page.The default size of the buffer is 8Kb.

Example of buffer attribute

```
<html>
<body>
  <%@ page buffer="16kb" %>
Today is: <%= new java.util.Date() %>
  </body>
</html>
```

6)language

The language attribute specifies the scripting language used in the JSP page. The default value is "java".

7)isELIgnored

We can ignore the Expression Language (EL) in jsp by the isELIgnored attribute. By default its value is false i.e. Expression Language is enabled by default. We see Expression Language later.

```
<%@ page isELIgnored="true" %>//Now EL will be ignored
```

8)isThreadSafe

- Servlet and JSP both are multithreaded.If you want to control this behaviour of JSP page, you can use isThreadSafe attribute of page directive.
- The value of isThreadSafe value is true.If you make it false, the web container will serialize the multiple requests, i.e. it will wait until the JSP finishes responding to a request before passing another request to it.If you make the value of isThreadSafe attribute like:

```
<%@ page isThreadSafe="false" %>
```

The web container in such a case, will generate the servlet as:

```
public class SimplePage_jsp extends HttpJspBase
  implements SingleThreadModel{
.......
}
```

9)errorPage

The errorPage attribute is used to define the error page, if exception occurs in the current page, it will be redirected to the error page.

Example of errorPage attribute

```
//index.jsp
<html>
<body>

<%@ page errorPage="myerrorpage.jsp" %>

 <%= 100/0 %>

</body>
</html>
```

10)isErrorPage

The isErrorPage attribute is used to declare that the current page is the error page.

Example of isErrorPage attribute

```
//myerrorpage.jsp
<html>
<body>
  <%@ page isErrorPage="true" %>
  Sorry an exception occured!<br/>
The exception is: <%= exception %>
  </body>
</html>
```

Jsp Include Directive

The include directive is used to include the contents of any resource it may be jsp file, html file or text file. The include directive includes the original content of the included resource at page translation time (the jsp page is translated only once so it will be better to include static resource).

Advantage of Include directive

Code Reusability

Syntax of include directive

<%@ include file="resourceName" %>

Example of include directive

In this example, we are including the content of the header.html file. To run this example you must create an header.html file.

```
<html>
<body>

<%@ include file="header.html" %>

Today is: <%= java.util.Calendar.getInstance().getTime() %>

</body>
</html>
```

The JSP taglib directive is used to define a tag library that defines many tags. We use the TLD (Tag Library Descriptor) file to define the tags. In the custom tag section we will use this tag so it will be better to learn it in custom tag.

Syntax JSP Taglib directive

```
<%@ taglib uri="uriofthetaglibrary" prefix="prefixoftaglibrary" %>
```

Example of JSP Taglib directive

In this example, we are using our tag named currentDate. To use this tag we must specify the taglib directive so the container may get information about the tag.

```
<html>
<body>

<%@ taglib uri="http://www.javatpoint.com/tags" prefix="mytag" %>

<mytag:currentDate/>

</body>
</html>
```

ACTION ELEMENTS

ACTION ELEMENTS

JSP Action Tags

- There are many JSP action tags or elements. Each JSP action tag is used to perform some specific tasks.
- The action tags are used to control the flow between pages and to use Java Bean.

Here is the list of JSP Actions:

- jsp:forward: is used for forwarding the request and response to other resources.
- jsp:include: is used for including another resource.
- jsp:useBean: is used for creating or locating bean objects.
- jsp:setProperty: is used for setting the value of property in the bean object.
- jsp:getProperty: is used for printing the value of the property of the bean.

- jsp:plugin: is used for embedding other components (applets).
- jsp:param: is used for setting the parameter for (forward or include) value.
- jsp:fallback: is used for printing the message if plugins are working.

jsp:forward action tag

The jsp:forward action tag is used to forward the request to another resource it may be jsp, html or another resource.

Syntax of jsp:forward action tag without parameter

```
<jsp:forward page="relativeURL | <%= expression %>" />
```

Syntax of jsp:forward action tag with parameter

```
<jsp:forward page="relativeURL | <%= expression %>">
<jsp:param name="parametername" value="parametervalue | <%=expression%>" />
</jsp:forward>
```

jsp:include action tag

- The jsp:include action tag is used to include the content of another resource it may be jsp, html or servlet.
- The jsp include action tag includes the resource at request time so it is better for dynamic pages because there might be changes in future.
- The jsp:include tag can be used to include static as well as dynamic pages.

Advantage of jsp:include action tag

Code reusability : We can use a page many times such as including header and footer pages in all pages. So it saves a lot of time.

Syntax of jsp:include action tag without parameter

```
<jsp:include page="relativeURL | <%= expression %>" />
```

Syntax of jsp:include action tag with parameter

```
<jsp:include page="relativeURL | <%= expression %>">
<jsp:param name="parametername" value="parametervalue | <%=expression%>" />
</jsp:include>
```

Example of jsp:include action tag without parameter

In this example, index.jsp file includes the content of the printdate.jsp file.

File: index.jsp

```
<h2>this is index page</h2>

<jsp:include page="printdate.jsp" />

<h2>end section of index page</h2>
```

File: printdate.jsp

```
<% out.print("Today is:"+java.util.Calendar.getInstance().getTime()); %>
```

Example of jsp:forward action tag without parameter

In this example, we are simply forwarding the request to the printdate.jsp file.

index.jsp

```
<html>
<body>
<h2>this is index page</h2>

<jsp:forward page="printdate.jsp" />
</body>
</html>
```

printdate.jsp

```
<html>
<body>
<% out.print("Today is:"+java.util.Calendar.getInstance().getTime()); %>
</body>
</html>
```

Example of jsp:forward action tag with parameter

In this example, we are forwarding the request to the printdate.jsp file with parameter and printdate.jsp file prints the parameter value with date and time.

index.jsp

```
<html>
<body>
<h2>this is index page</h2>
  <jsp:forward page="printdate.jsp" >
<jsp:param name="name" value="javatpoint.com" />
</jsp:forward>
  </body>
</html>
```

printdate.jsp

```
<html>
<body>

<% out.print("Today is:"+java.util.Calendar.getInstance().getTime()); %>
<%= request.getParameter("name") %>

</body>
</html>
```

JavaBean

A JavaBean is a Java class that should follow the following conventions:

- It should have a no-arg constructor.
- It should be Serializable.
- It should provide methods to set and get the values of the properties, known as getter and setter methods.

Why use JavaBean?

According to Java white paper, it is a reusable software component. A bean encapsulates many objects into one object so that we can access this object from multiple places. Moreover, it provides easy maintenance.

Simple example of JavaBean class

//Student.java

```
package com.training.model;
public class Student {
private int sid;
private String name;
//constructor with super method to access parent properties
public Student() {
   super();
 }
//constructor with fields to access data
public Student(int sid, String name) {
   super();
   this.sid = sid;
   this.name = name;
}
```

```
//getter and setter method to return and set data
public int getSid() {
  return sid;
}
public void setSid(int sid) {
  this.sid = sid;
}
public String getName() {
  return name;
}
public void setName(String name) {
  this.name = name;
}
}
```

How to access the JavaBean class?

To access the JavaBean class, we should use getter and setter methods.

```
ppackage com.training;

import java.io.IOException;
import com.training.model.Student;
import jakarta.servlet.ServletException;
import jakarta.servlet.annotation.WebServlet;
import jakarta.servlet.http.HttpServlet;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;
```

```
@WebServlet("/students")
public class StudentController extends HttpServlet {


    protected void doPost(HttpServletRequest request,
HttpServletResponse response)
            throws ServletException, IOException {
        String sidStr = request.getParameter("sid");
        String name = request.getParameter("sname");
```

```
        try {
            int sid = Integer.parseInt(sidStr);

            Student student = new Student();
            student.setSid(sid);
            student.setName(name);

            System.out.println("Student Details:");
            System.out.println("ID: " + student.getSid());
            System.out.println("Name: " + student.getName());

            response.getWriter().println("Student details saved successfully!");

        } catch (NumberFormatException e) {
            response.getWriter().println("Invalid ID entered. Please enter a number.");
        }
    }
}
```

JavaBean Properties

A JavaBean property is a named feature that can be accessed by the user of the object. The feature can be of any Java data type, containing the classes that you define.

A JavaBean property may be read, write, read-only, or write-only. JavaBean features are accessed through two methods in the JavaBean's implementation class:

1. getPropertyName ()

For example, if the property name is firstName, the method name would be getFirstName() to read that property. This method is called the accessor.

2. setPropertyName ()

For example, if the property name is firstName, the method name would be setFirstName() to write that property. This method is called the mutator.

Advantages of JavaBean

The following are the advantages of JavaBean:/p>

- The JavaBean properties and methods can be exposed to another application.
- It provides an easiness to reuse the software components.

Disadvantages of JavaBean

The following are the disadvantages of JavaBean:

- JavaBeans are mutable. So, it can't take advantages of immutable objects.
- Creating the setter and getter method for each property separately may lead to the boilerplate code.

jsp:useBean action tag

The jsp:useBean action tag is used to locate or instantiate a bean class. If bean object of the Bean class is already created, it doesn't create the bean depending on the scope. But if object of bean is not created, it instantiates the bean.

Syntax of jsp:useBean action tag

```
<jsp:useBean id= "instanceName" scope= "page | request | session | application"
class= "packageName.className" type= "packageName.className"
beanName="packageName.className | <%= expression >" >
</jsp:useBean>
```

Attributes and Usage of jsp:useBean action tag

1. **id:** is used to identify the bean in the specified scope.

2. **scope:** represents the scope of the bean. It may be page, request, session or application. The default scope is page.

- **page:** specifies that you can use this bean within the JSP page. The default scope is page.
- **request:** specifies that you can use this bean from any JSP page that processes the same request. It has wider scope than page.
- **session:** specifies that you can use this bean from any JSP page in the same session whether processes the same request or not. It has wider scope than request.
- **application:** specifies that you can use this bean from any JSP page in the same application. It has wider scope than session.

3. **class:** instantiates the specified bean class (i.e. creates an object of the bean class) but it must have no-arg or no constructor and must not be abstract.
4. **type:** provides the bean a data type if the bean already exists in the scope. It is mainly used with class or beanName attribute. If you use it without class or beanName, no bean is instantiated.
5. **beanName:** instantiates the bean using the java.beans.Beans.instantiate() method.

Simple example of jsp:useBean action tag

In this example, we are simply invoking the method of the Bean class.

```
<%@ page language="java" contentType="text/html; charset=UTF-8" pageEncoding="UTF-8"%>
<!DOCTYPE html>
<html>
<head>
<meta charset="UTF-8">
<title>Student Registration</title>
</head>
<body>
<h2>Enter Student Details</h2>
<form action="students" method="post">
   ID: <input type="text" name="sid" placeholder="Enter Student ID"> <br><br>
   Name: <input type="text" name="sname" placeholder="Enter Student Name"> <br><br>
   <input type="submit" value="Submit">
</form>
</body>
</html>
```

jsp:setProperty and jsp:getProperty action tags

- The setProperty and getProperty action tags are used for developing web application with Java Bean. In web devlopment, bean class is mostly used because it is a reusable software component that represents data.
- The jsp:setProperty action tag sets a property value or values in a bean using the setter method.

Syntax of jsp:setProperty action tag

```
<jsp:setProperty name="instanceOfBean" property= "*"   |
property="propertyName" param="parameterName"  |
property="propertyName" value="{ string | <%= expression %>}"
/>
```

Example of jsp:setProperty action tag if you have to set all the values of incoming request in the bean

```
<jsp:setProperty name="bean" property="*" />
```

Example of jsp:setProperty action tag if you have to set value of the incoming specific property

```
<jsp:setProperty name="bean" property="username" />
```

Example of jsp:setProperty action tag if you have to set a specific value in the property

```
<jsp:setProperty name="bean" property="username" value="Kumar" />
```

jsp:getProperty action tag

The jsp:getProperty action tag returns the value of the property.

Syntax of jsp:getProperty action tag

```
<jsp:getProperty name="instanceOfBean" property="propertyName" />
```

Simple example of jsp:getProperty action tag

```
<jsp:getProperty name="obj" property="name" />
```

Example of bean development in JSP

In this example there are 3 pages:

- **student.jsp** — for input of values
- **home.jsp** — file that sets the incoming values to the bean object and prints all the values
- **update.jsp** — file that allows updating the values and stores the updated values using the bean

STUDENT.JSP

```
<%@ page language="java" contentType="text/html; charset=UTF-8" pageEncoding="UTF-8"%>
<jsp:useBean id="student" class="com.training.model.Student" scope="session" />
<!DOCTYPE html>
<html>
<head>
<meta charset="UTF-8">
<title>Student Input</title>
</head>
<body>
<h2>Enter Student Details</h2>
<form action="home.jsp" method="post">
   ID: <input type="text" name="sid" placeholder="Enter ID"> <br><br>
   Name: <input type="text" name="name" placeholder="Enter Name"> <br><br>
   <input type="submit" value="Submit">
</form>
</body>
</html>
```

HOME.JSP

```
<%@ page language="java" contentType="text/html; charset=UTF-8" pageEncoding="UTF-8"%>
<jsp:useBean id="student" class="com.training.model.Student" scope="session" />
<jsp:setProperty name="student" property="sid" param="sid" />
<jsp:setProperty name="student" property="name" param="name" />
<!DOCTYPE html>
<html>
<head>
<meta charset="UTF-8">
<title>Home - Student Details</title>
</head>
<body>
<h2>Student Details Saved</h2>
ID: <jsp:getProperty name="student" property="sid" /> <br>
Name: <jsp:getProperty name="student" property="name" /> <br><br>
<form action="update.jsp" method="post">
   <input type="submit" value="Update Details">
</form>
<h1>hello , <jsp:getProperty name='student' property='name' /></h1>
</body>
</html>
```

UPDATE.JSP

```
<%@ page language="java" contentType="text/html; charset=UTF-8" pageEncoding="UTF-8"%>
<jsp:useBean id="student" class="com.training.model.Student" scope="session" />
<!DOCTYPE html>
<html>
<head>
<meta charset="UTF-8">
<title>Update Student</title>
</head>
<body>
<h2>Update Student Details</h2>
<form action="home.jsp" method="post">
   ID: <input type="text" name="sid" value="<jsp:getProperty name='student' property='sid' />"> <br><br>
   Name: <input type="text" name="name" value="<jsp:getProperty name='student' property='name' />">
<br><br>
   <input type="submit" value="Update and Save">
</form>
<h1>hello , <jsp:getProperty name='student' property='name' /></h1>
</body>
</html
```

JSP EXPRESSION LANGUAGE

Expression Language (EL) in JSP

- The Expression Language (EL) simplifies the accessibility of data stored in the Java Bean component, and other objects like request, session, application etc.
- There are many implicit objects, operators and reserve words in EL.
- It is the newly added feature in JSP technology version 2.0.

Syntax for Expression Language (EL)

${ expression }

IMPLICIT OBJECTS

JSP Implicit Objects are **predefined variables** provided by the JSP container.

They are automatically created and available for direct use in JSP pages **without explicit declaration or initialization**.

These objects help in performing tasks like handling requests, responses, sessions, application data, and exceptions.

LIST OF IMPLICIT OBJECTS

OBJECT	USE
out	Used to send content to the client. Example: out.println("Hello");
request	Represents the HTTP request; used to get request parameters and attributes. Example: request.getParameter("name");
response	Represents the HTTP response; used to modify response data. Example: response.getWriter();
config	Provides servlet configuration information.

application	Refers to the ServletContext object for sharing data across the application.
session	Represents the user session object; used to store user-specific data.
pageContext	Provides access to all the namespaces associated with a JSP page.
page	Refers to the current JSP page (equivalent to this in Java).
exception	Used for exception handling in JSP error pages.

EXAMPLE CODE FOR IMPLICIT OBJECTS

```
<%@ page language="java" contentType="text/html; charset=ISO-8859-1"
pageEncoding="ISO-8859-1"%>
<!DOCTYPE html>
<html>
<head>
   <title>JSP Implicit Objects Example</title>
</head>
<body>
<%
   out.println("Request Method: " + request.getMethod() + "<br/>");  // JSP
Request
   out.println("Request URI: " + request.getRequestURI() + "<br/>");  // JSP
Request
   out.println("Response Buffer Size: " + response.getBufferSize() + "<br/>");
// JSP Response
```

```
    out.println("Application Server Info: " + application.getServerInfo() + "<br/>");
// JSP Application
   out.println("Session ID: " + session.getId() + "<br/>");  // JSP Session
   out.println("Page Context: " + pageContext.getServletContext() + "<br/>");  //
JSP PageContext
   out.println("Page: " + page + "<br/>");  // JSP Page

%>
</body>
</html>
```

CREATING CUSTOM TAGS

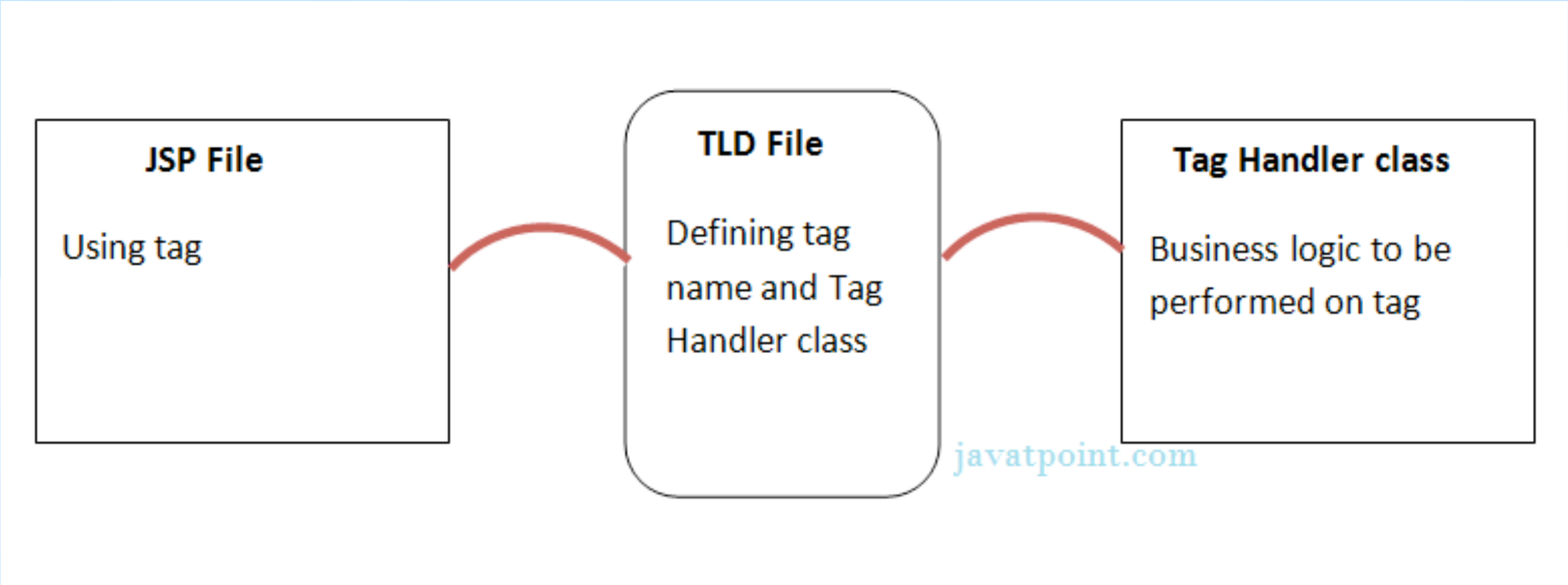
Example of JSP Custom Tag

In this example, we are going to create a custom tag that prints the current date and time. We are performing action at the start of tag.

For creating any custom tag, we need to follow following steps:

- Create the Tag handler class and perform action at the start or at the end of the tag.
- Create the Tag Library Descriptor (TLD) file and define tags
- Create the JSP file that uses the Custom tag defined in the TLD file

Understanding flow of custom tag in jsp



1) Create the Tag handler class

- To create the Tag Handler, we are inheriting the TagSupport class and overriding its method doStartTag().To write data for the jsp, we need to use the JspWriter class.
- The PageContext class provides getOut() method that returns the instance of JspWriter class. TagSupport class provides instance of pageContext bydefault.

```
package com.javatpoint.sonoo;
import java.util.Calendar;
import javax.servlet.jsp.JspException;
import javax.servlet.jsp.JspWriter;
import javax.servlet.jsp.tagext.TagSupport;
public class MyTagHandler extends TagSupport{
```

```
public int doStartTag() throws JspException {
   JspWriter out=pageContext.getOut();//returns the instance of JspWriter
   try{
    out.print(Calendar.getInstance().getTime());//printing date and time using JspWriter
   }catch(Exception e){System.out.println(e);}
   return SKIP_BODY;//will not evaluate the body content of the tag
}
}
```

2) Create the TLD file

Tag Library Descriptor (TLD) file contains information of tag and Tag Hander classes. It must be contained inside the WEB-INF directory.

```
<?xml version="1.0" encoding="ISO-8859-1" ?>
<!DOCTYPE taglib
      PUBLIC "-//Sun Microsystems, Inc.//DTD JSP Tag Library 1.2//EN"
   "http://java.sun.com/j2ee/dtd/web-jsptaglibrary_1_2.dtd">
  <taglib>
 <tlib-version>1.0</tlib-version>
  <jsp-version>1.2</jsp-version>
  <short-name>simple</short-name>
  <uri>http://tomcat.apache.org/example-taglib</uri>
<tag>
<name>today</name>
<tag-class>com.javatpoint.sonoo.MyTagHandler</tag-class>
</tag>
</taglib>
```

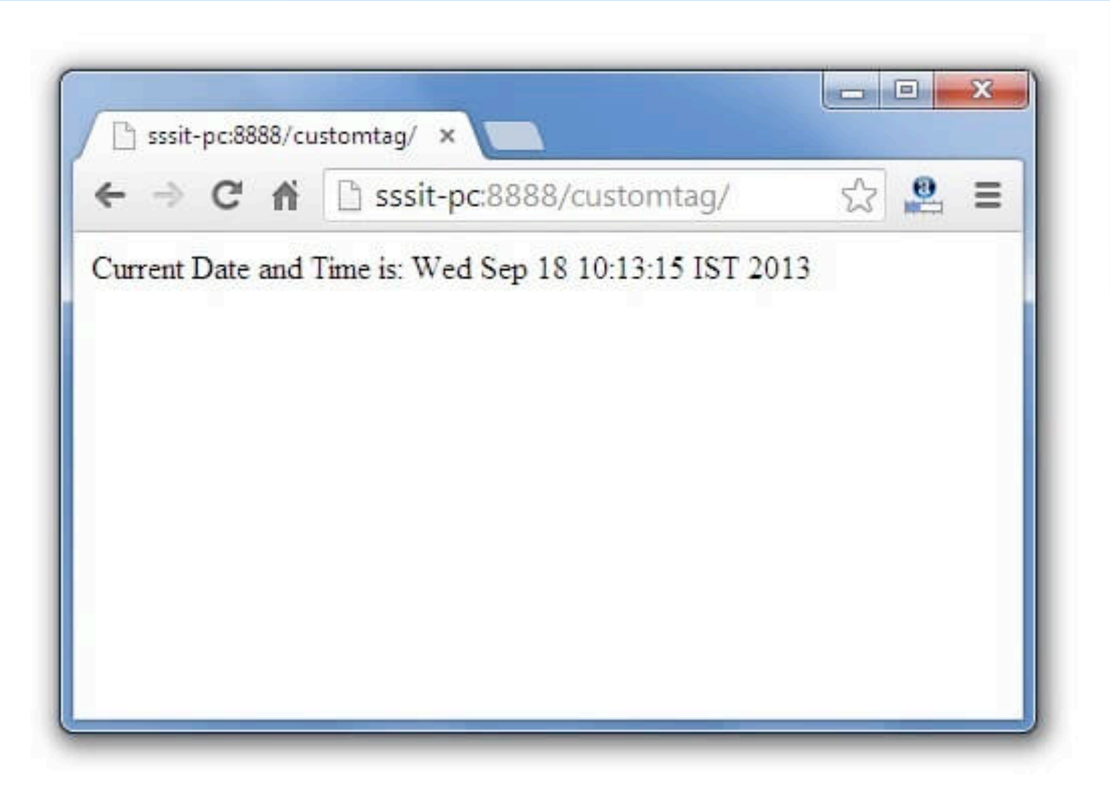
3) Create the JSP file

- Let's use the tag in our jsp file. Here, we are specifying the path of tld file directly. But it is recommended to use the uri name instead of full path of tld file. We will learn about uri later.
- It uses taglib directive to use the tags defined in the tld file.

File: index.jsp

```
<%@ taglib uri="WEB-INF/mytags.tld" prefix="m" %>
Current Date and Time is: <m:today/>
```

Output


sssit-pc:8888/customtag/
sssit-pc:8888/customtag/
Current Date and Time is: Wed Sep 18 10:13:15 IST 2013

Attributes in JSP Custom Tag

There can be defined too many attributes for any custom tag. To define the attribute, you need to perform two tasks:

- Define the property in the TagHandler class with the attribute name and define the setter method
- Define the attribute element inside the tag element in the TLD file

```
<m:cube number="4"></m:cube>
```

Here m is the prefix, cube is the tag name and number is the attribute.

Simple example of attribute in JSP Custom Tag

In this example, we are going to use the cube tag which return the cube of any given number. Here, we are defining the number attribute for the cube tag. We are using the three file here:

- index.jsp
- CubeNumber.java
- mytags.tld

index.jsp

```
<%@ taglib uri="WEB-INF/mytags.tld" prefix="m" %>
Cube of 4 is: <m:cube number="4"></m:cube>
```

CubeNumber.java

```
package com.javatpoint.taghandler;
import javax.servlet.jsp.JspException;
import javax.servlet.jsp.JspWriter;
import javax.servlet.jsp.tagext.TagSupport;

public class CubeNumber extends TagSupport{
private int number;

public void setNumber(int number) {
   this.number = number;
}
```

```
public int doStartTag() throws JspException {
    JspWriter out=pageContext.getOut();
    try{
        out.print(number*number*number);
    }catch(Exception e){e.printStackTrace();}

    return SKIP_BODY;
}
}
```

mytags.tld

```
<?xml version="1.0" encoding="ISO-8859-1" ?>
<!DOCTYPE taglib
      PUBLIC "-//Sun Microsystems, Inc.//DTD JSP Tag Library 1.2//EN"
      "http://java.sun.com/j2ee/dtd/web-jsptaglibrary_1_2.dtd">

<taglib>
  <tlib-version>1.0</tlib-version>
  <jsp-version>1.2</jsp-version>
  <short-name>simple</short-name>
  <uri>http://tomcat.apache.org/example-taglib</uri>
  <description>A simple tab library for the examples</description>
```

```
<tag>
   <name>cube</name>
   <tag-class>com.javatpoint.taghandler.CubeNumber</tag-class>
   <attribute>
   <name>number</name>
   <required>true</required>
   </attribute>
  </tag>
</taglib>
```

Output

Cube of 4 is: 64

Iteration using JSP Custom Tag

- We can iterate the body content of any tag using the doAfterBody() method of IterationTag interface.
- Here we are going to use the TagSupport class which implements the IterationTag interface. For iterating the body content, we need to use the EVAL_BODY_AGAIN constant in the doAfterBody() method.

Example of Iteration using JSP Custom Tag

In this example, we are going to use the attribute in the custom tag, which returns the power of any given number. We have created three files here

- index.jsp
- PowerNumber.java
- mytags.tld

index.jsp

```
<%@ taglib uri="WEB-INF/mytags.tld" prefix="m" %>
  3 ^ 5 = <m:power number="3" power="5">
body
</m:power>
```

PowerNumber.java

```
package com.javatpoint.taghandler;
import javax.servlet.jsp.JspException;
import javax.servlet.jsp.JspWriter;
import javax.servlet.jsp.tagext.TagSupport;
  public class PowerNumber extends TagSupport{
private int number;
private int power;
private static int counter;
private static int result=1;
```

```
public void setPower(int power) {
    this.power = power;
}

public void setNumber(int number) {
    this.number = number;
}

public int doStartTag() throws JspException {
    return EVAL_BODY_INCLUDE;
}
```

```
public int doAfterBody() {
   counter++;
   result *= number;
   if (counter==power)
     return SKIP_BODY;
   else
     return EVAL_BODY_AGAIN;
  }
public int doEndTag() throws JspException {
   JspWriter out=pageContext.getOut();
   try{
      out.print(result);
   }catch(Exception e){e.printStackTrace();}
   return EVAL_PAGE;
}
}
```

mytags.tld

```
<?xml version="1.0" encoding="ISO-8859-1" ?>
<!DOCTYPE taglib
      PUBLIC "-//Sun Microsystems, Inc.//DTD JSP Tag Library 1.2//EN"
      "http://java.sun.com/j2ee/dtd/web-jsptaglibrary_1_2.dtd">

<taglib>
  <tlib-version>1.0</tlib-version>
  <jsp-version>1.2</jsp-version>
  <short-name>simple</short-name>
  <uri>http://tomcat.apache.org/example-taglib</uri>
  <description>A simple tab library for the examples</description>
```

```
 <tag>
   <name>power</name>
   <tag-class>com.javatpoint.taghandler.PowerNumber</tag-class>
   <attribute>
   <name>number</name>
   <required>true</required>
   </attribute>
   <attribute>
   <name>power</name>
   <required>true</required>
   </attribute>
  </tag>
</taglib>
```

Looping using Iteration Tag (creating tag for loop)

Let's create a loop tag that iterates the body content of this tag.

File: index.jsp

```
<!DOCTYPE html PUBLIC "-//W3C//DTD HTML 4.01 Transitional//EN"
 "http://www.w3.org/TR/html4/loose.dtd">
<html>
<head>
<title>Insert title here</title>
</head>
<body>
  <%@taglib prefix="m" uri="sssuri" %>
<m:loop end="5" start="1">
<p>My Name is khan</p>
</m:loop>
</body>
</html>
```

File: mytags.tld

```
<?xml version="1.0" encoding="ISO-8859-1" ?>
<!DOCTYPE taglib
      PUBLIC "-//Sun Microsystems, Inc.//DTD JSP Tag Library 1.2//EN"
      "http://java.sun.com/j2ee/dtd/web-jsptaglibrary_1_2.dtd">
<taglib>
 <tlib-version>1.0</tlib-version>
 <jsp-version>1.2</jsp-version>
 <short-name>abc</short-name>
<uri>sssuri</uri>
<tag>
  <name>loop</name>
  <tag-class>com.javatpoint.customtag.Loop</tag-class>
```

```
<attribute>
  <name>start</name>
  <required>true</required>
  </attribute>

  <attribute>
  <name>end</name>
  <required>true</required>
  </attribute>
</tag>

</taglib>
```

File: Loop.java

```
package com.javatpoint.customtag;
import javax.servlet.jsp.JspException;
import javax.servlet.jsp.tagext.TagSupport;

public class Loop extends TagSupport{
    private int start=0;
    private int end=0;

    public void setStart(int start) {
        this.start = start;
    }
```

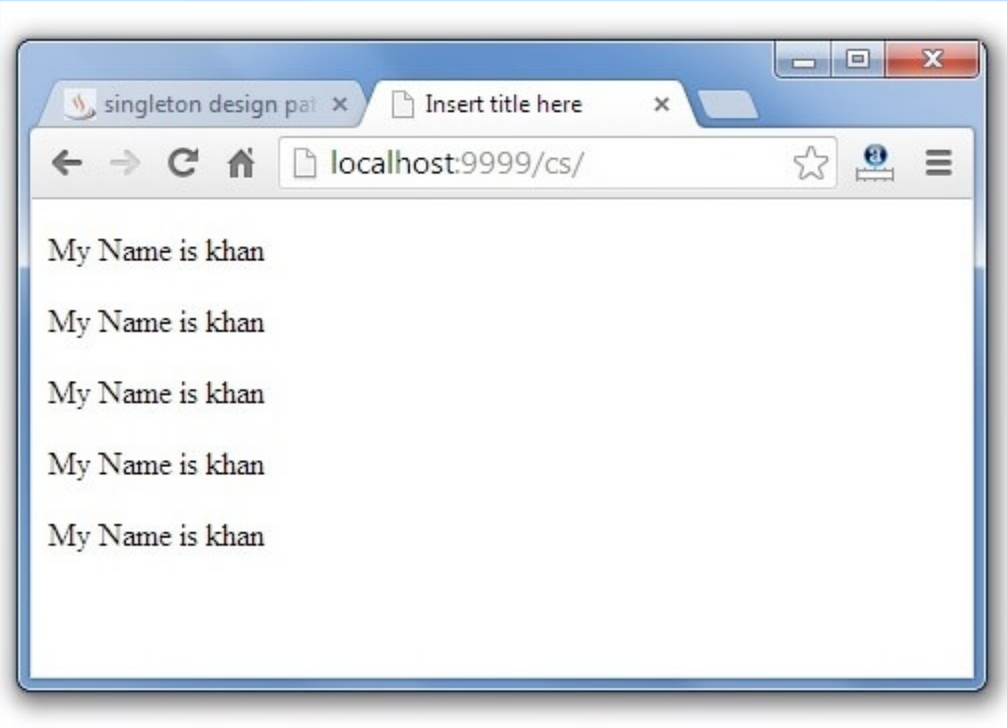
```
public void setEnd(int end) {
    this.end = end;
  }
  @Override
  public int doStartTag() throws JspException {
    return EVAL_BODY_INCLUDE;
  }
  @Override
  public int doAfterBody() throws JspException {
    if(start<end){
      start++;
      return EVAL_BODY_AGAIN;
    }else{
    return SKIP_BODY;
   }
  }
```

File: web.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<web-app xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xmlns="http://java.sun.com/xml/ns/javaee"
 xmlns:web="http://java.sun.com/xml/ns/javaee/web-app_2_5.xsd"
xsi:schemaLocation="http://java.sun.com/xml/ns/javaee
http://java.sun.com/xml/ns/javaee/web-app_3_0.xsd" id="WebApp_ID" version="3.0">
<jsp-config>
<taglib>
<taglib-uri>sssuri</taglib-uri>
<taglib-location>/WEB-INF/mytags.tld</taglib-location>
</taglib>
</jsp-config>

</web-app>
```

Output


singleton design pat
Insert title here
localhost:9999/cs/
My Name is khan
My Name is khan
My Name is khan
My Name is khan
My Name is khan

Custom URI in JSP Custom Tag

We can use the custom URI, to tell the web container about the tld file. In such case, we need to define the taglib element in the web.xml. The web container gets the information about the tld file from the web.xml file for the specified URI.

Example to use custom URI in JSP Custom Tag

- index.jsp
- web.xml
- mytags.tld
- PrintDate.java

index.jsp

```
<%@ taglib uri="mytags" prefix="m" %>
Today is: <m:today></m:today>
```

web.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE web-app
  PUBLIC "-//Sun Microsystems, Inc.//DTD Web Application 2.3//EN"
  "http://java.sun.com/dtd/web-app_2_3.dtd">
<web-app>
<jsp-config>
<taglib>
<taglib-uri>mytags</taglib-uri>
<taglib-location>/WEB-INF/mytags.tld</taglib-location>
</taglib>
</jsp-config>
</web-app>
```

mytags.tld

```
<?xml version="1.0" encoding="ISO-8859-1" ?>
<!DOCTYPE taglib
      PUBLIC "-//Sun Microsystems, Inc.//DTD JSP Tag Library 1.2//EN"
      "http://java.sun.com/j2ee/dtd/web-jsptaglibrary_1_2.dtd">
<taglib>
  <tlib-version>1.0</tlib-version>
  <jsp-version>1.2</jsp-version>
  <short-name>simple</short-name>
  <uri>mytags</uri>
  <description>A simple tab library for the examples</description>
  <tag>
    <name>today</name>
    <tag-class>com.javatpoint.taghandler.PrintDate</tag-class>
  </tag>
</taglib>
```

PrintDate.java

```
package com.javatpoint.taghandler;
import javax.servlet.jsp.JspException;
import javax.servlet.jsp.JspWriter;
import javax.servlet.jsp.tagext.TagSupport;
public class PrintDate extends TagSupport{
public int doStartTag() throws JspException {
   JspWriter out=pageContext.getOut();
   try{
      out.print(java.util.Calendar.getInstance().getTime());
   }catch(Exception e){e.printStackTrace();}
   return SKIP_BODY;
   }
}
```

Working with XML

How to work with XML in JSP Application

- In this article, I am going to discuss How to work with XML in JSP Application. Please read our previous article where we discussed How to interact with a Database in JSP Application.

Overview of XML:

- XML stands for Extensible Markup Language. It is derived from Standard Generalized Markup Language (SGML). It is a text-based markup language.

- It is a platform-independent language. It is also a language-independent language. XML is used to carry the data but not to display the data.

What is XML?

- XML is a markup language like HTML which is used to store and transport the data. XML is self-descriptive which contains sender information, receiver information, heading, and a message body.
- The XML does not DO anything as it is just information wrapped in tags.

Why we need XML?

- XML is self-describing where the structure of data is embedded with the data; so, there is no need to pre-build the structure to store the data; it is understood by the XML. XML document is “well-formed” and properly marked up.

XML versus HTML :

- In HTML, the syntax and semantics of the document are defined whereas HTML is used to create a visible presentation to the user.
- HTML documents are not well-formed whereas XML documents are well-formed.
- HTML displays the data whereas XML stores and transports the data.
- In HTML tags are predefined whereas in XML the tags are user-defined

Advantages of XML

- XML documents are plain text which can be edit and viewed from any simple text editor.

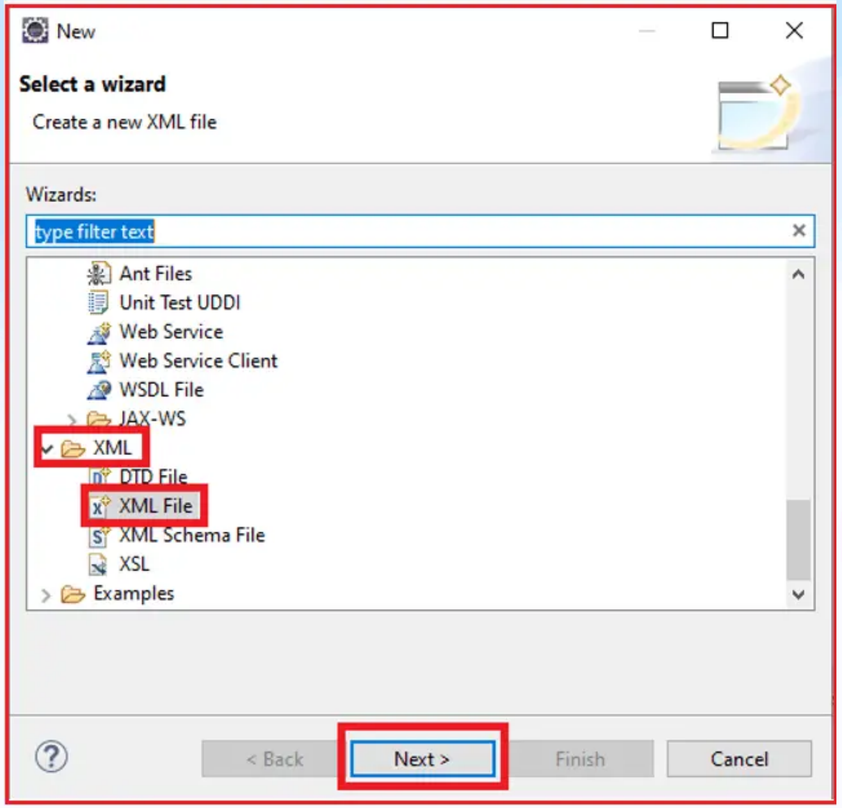
- XML documents have a tree structure that is very simple to understand.

- XML documents are platform-independent.

- XML is Unicode-based which takes less space and can be transmitted efficiently.

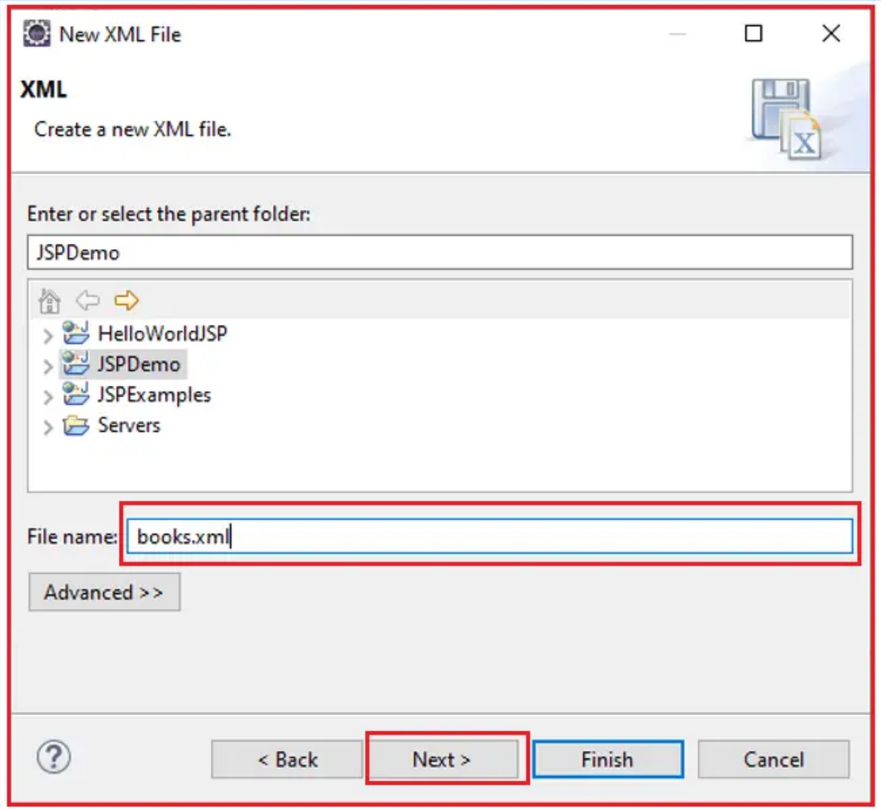
How to Create XML in JSP Application?

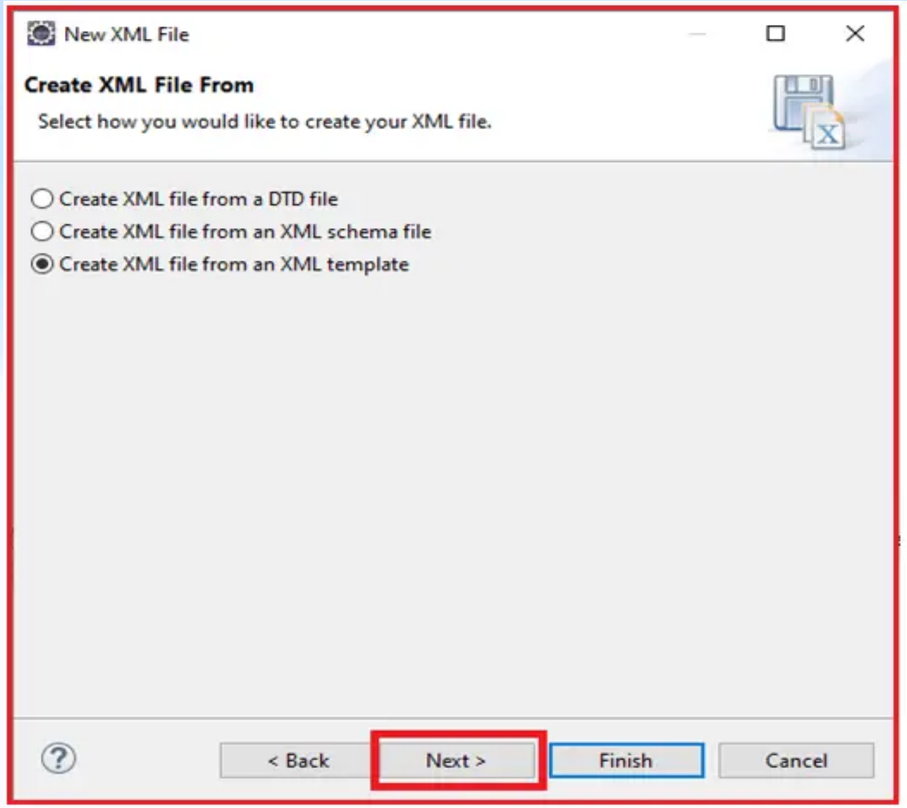
Step1: Right-click on the project. Go to New -> Other



How to Create XML in JSP Application?

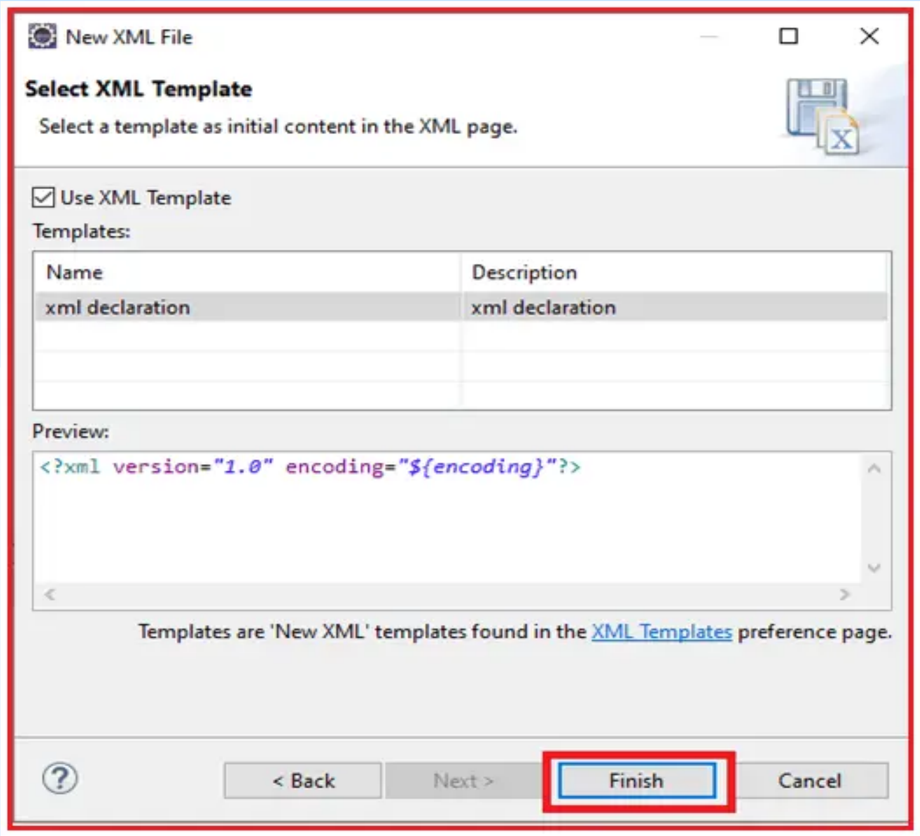
- Give a proper file name and click Next.
- Select how you want to create your XML file and click Next.

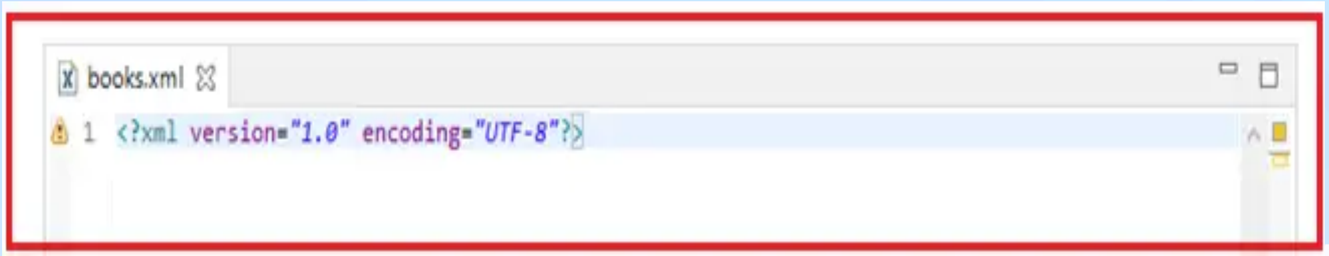




How to Create XML in JSP Application?

- Select the template and click Finish.
- Your XML File is created successfully.





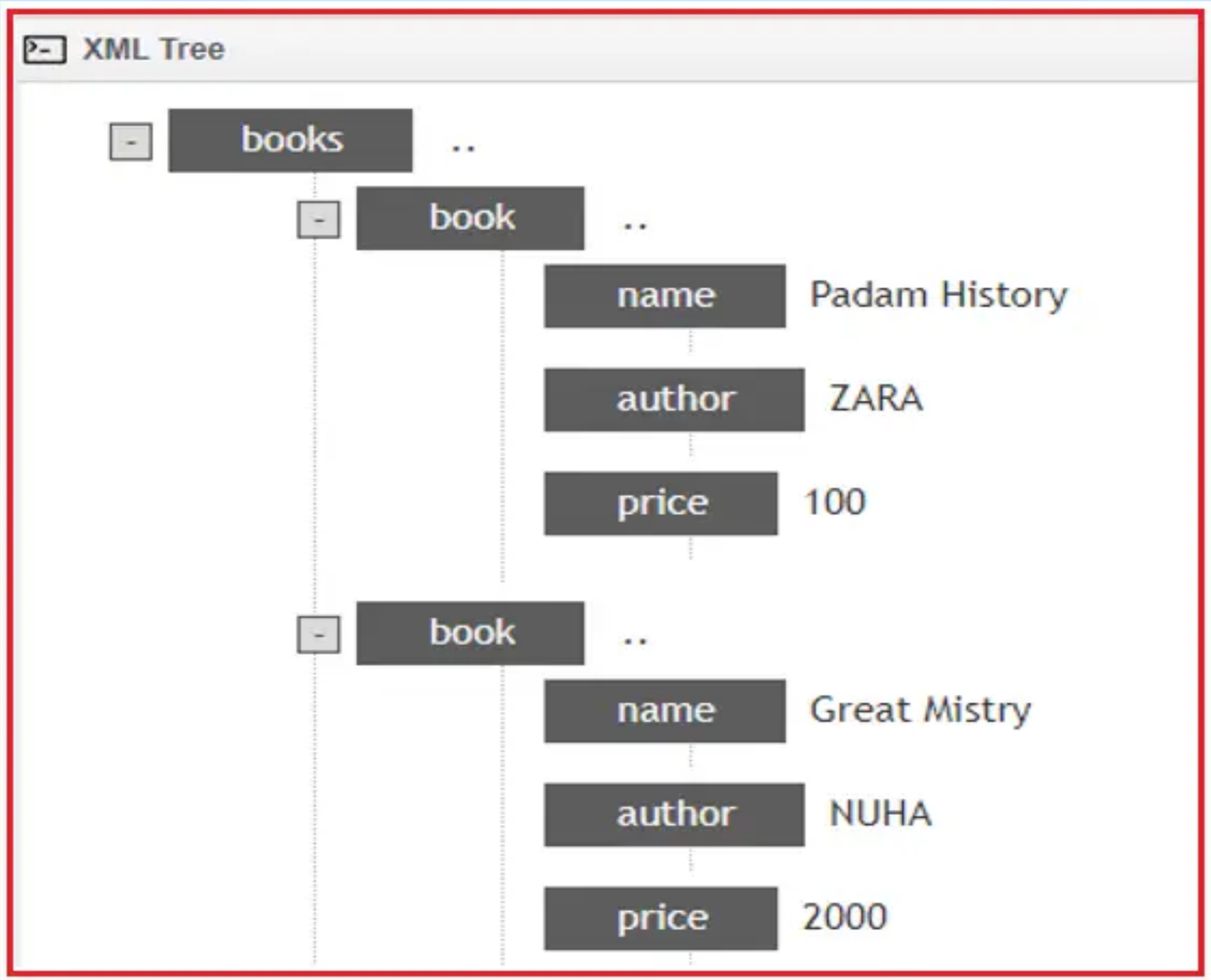
Presenting XML Documents :

- Let’s write an XML document that is presenting a book. Here we have one root element and every element has a start tag and a close tag. The first line in the example states the XML version number which is 1.

Boooks.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<books>
 <book>
  <name>Padam History</name>
  <author>ZARA</author>
  <price>100</price>
 </book>
 <book>
  <name>Great Mistry</name>
  <author>NUHA</author>
  <price>2000</price>
 </book>
</books>
```

The Tree Structure of the above XML file is:



Now load the books.xml document in a browser that supports XML, you will get the following output:

```
  <?xml version="1.0" encoding="UTF-8"?>
- <books>
  - <book>
      <name>Padam History</name>
      <author>ZARA</author>
      <price>100</price>
    </book>
  - <book>
      <name>Great Mistry</name>
      <author>NUHA</author>
      <price>2000</price>
    </book>
  </books>
```

Sending XML from a JSP / Generating XML from JSP :

- We can send or generate XML documents using JSP like HTML. It will easily generate a response containing the books.xml document. Here, we need to send the content type of our page to text/XML. Use <%@page%> tag to set the content type.

Syntax: <%@ page contentType = “text/xml” %>

```
<?xml version="1.0" encoding="UTF-8"?>
<books>
 <book>
  <name>Padam History</name>
  <author>ZARA</author>
  <price>100</price>
 </book>
</books>
```

Processing XML in JSP Application :

To process XML using JSP, copy the following two XML and XPath libraries into your “Tomcat Installation Directory/lib” folder:

- XercesImpl.jar
- xalan.jar

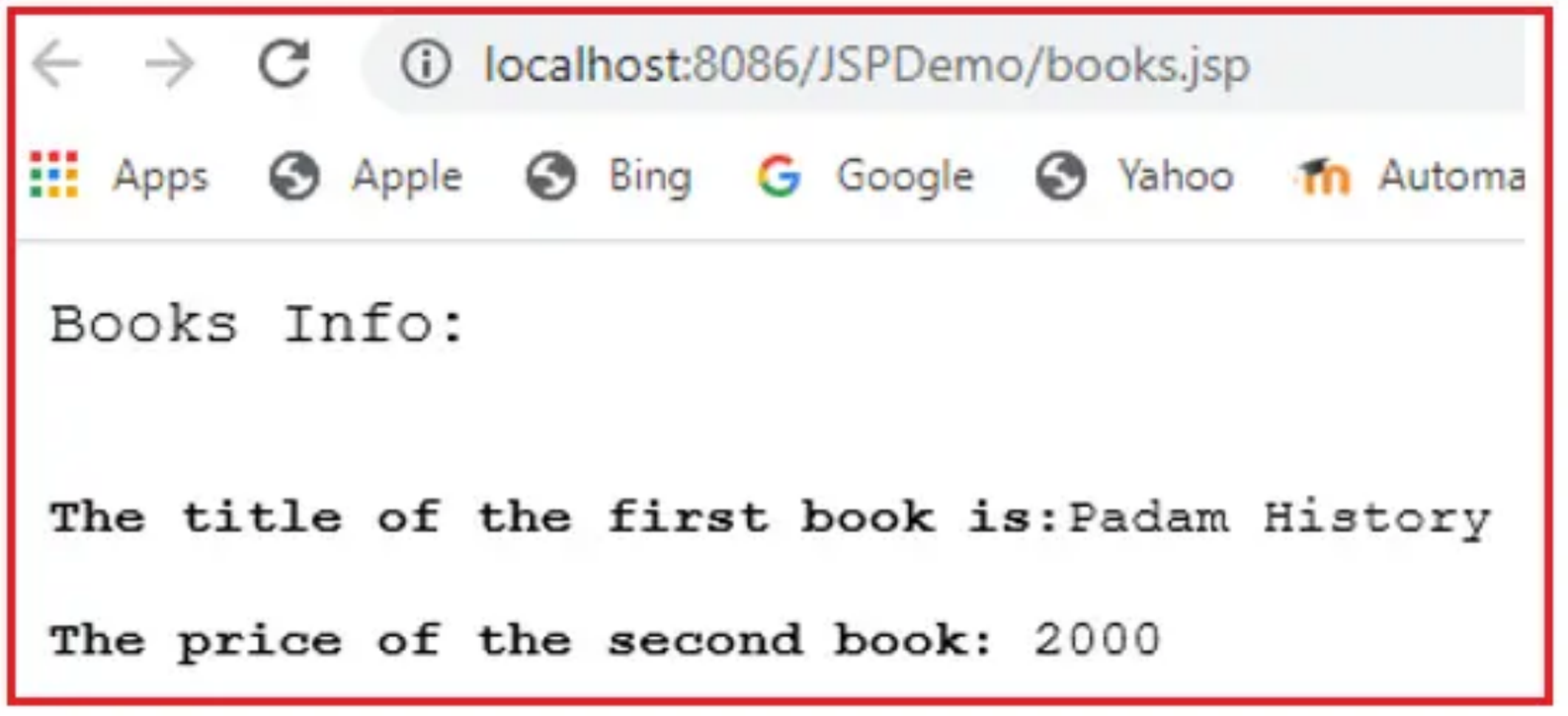
Example

- In this example, books.xml is the XML file that we are processing using books.jsp file.

books.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<books>
 <book>
  <name>Padam History</name>
  <author>ZARA</author>
  <price>100</price>
 </book>
 <book>
  <name>Great Mistry</name>
  <author>NUHA</author>
  <price>2000</price>
 </book>
</books>
```

Output :


localhost:8086/JSPDemo/books.jsp
Apps
Apple
Bing
Google
Yahoo
Automa
Books Info:
The title of the first book is:Padam History
The price of the second book: 2000

Formatting XML with JSP Application

- Formation of an XML file with JSP file can be done using a transformation language called Extensible Stylesheet Language Transformation (XSLT) which is a part of XSL. It allows you to write XML vocabulary for specifying formatting semantics.

Example

- In this example, style.xsl is an XSL stylesheet that performs the required transformation for the books element which generates HTML markup and extracts data from the elements in the books.xml document.

books.jsp

```
<%@ taglib prefix="c" uri="http://java.sun.com/jsp/jstl/core"%>
<%@ taglib prefix="x" uri="http://java.sun.com/jsp/jstl/xml"%>
<html>
<head>
<title>JSTL x:transform Tags</title>
</head>
<body>
 <h3>Books Info:</h3>
 <c:set var="xmltext">
  <books> <book> <name>Padam History</name> <author>ZARA</author>
  <price>100</price> </book> <book> <name>Great Mistry</name> <author>NUHA</author>
  <price>2000</price> </book> </books>
 </c:set>
 <c:import url="http://localhost:8086/JSPDemo/style.xsl" var="xslt" />
 <x:transform xml="${xmltext}" xslt="${xslt}" />
</body>
```

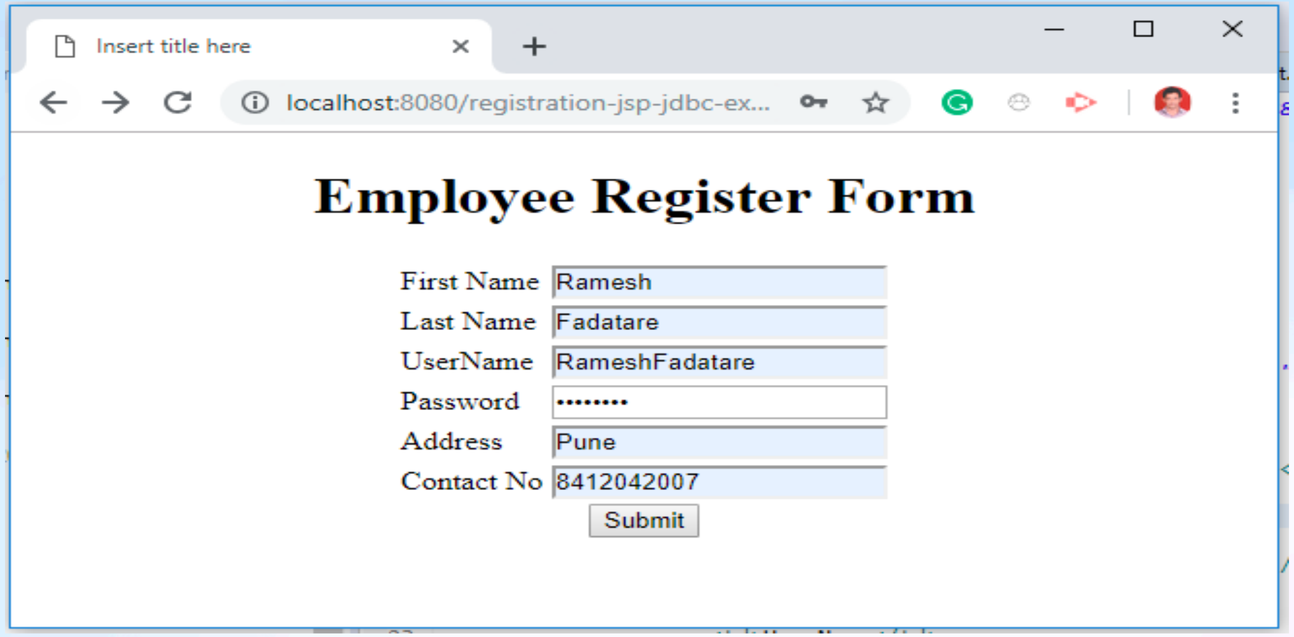
Output :

Padam History	ZARA	100
Great Mistry	NUHA	2000

Forms and Session management

Registration Form using JSP + Servlet :

Below diagram shows the Employee Registration JSP page:



- Let me list out the tools and technologies that I have used to develop this application.

Tools and technologies used

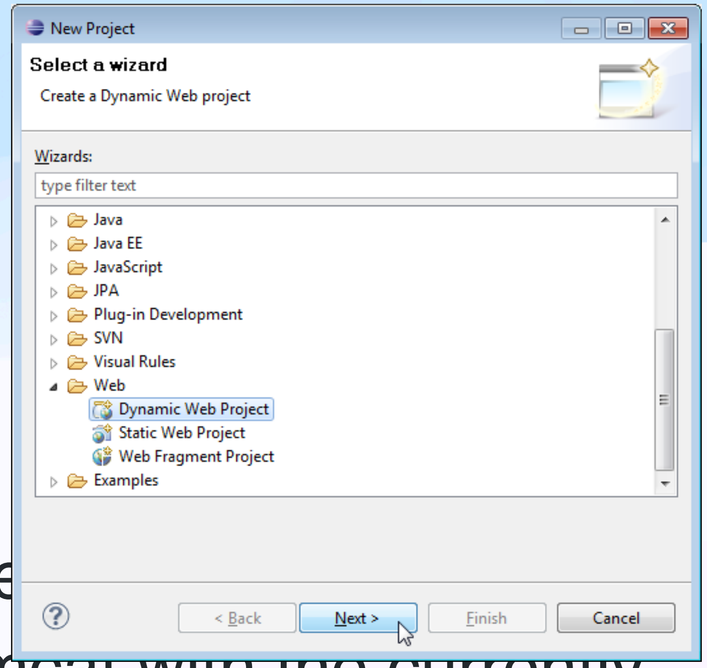
- JSP - 2.2 +
- IDE - STS/Eclipse Neon.3
- JDK - 1.8 or later
- Apache Tomcat - 8.5
- JSTL - 1.2.1
- Servlet API - 2.5
- MySQL - mysql-connector-java-8.0.13.jar

Development Steps

- Create an Eclipse Dynamic Web Project
- Add Dependencies
- Project Structure
- MySQL Database Setup
- Create a JavaBean - Employee.java
- Create a EmployeeDao.java
- Create a EmployeeServlet.java
- Create a employeeregister.jsp
- Create a employeedetail.jsp
- Demo

Create an Eclipse Dynamic Web Project

- To create a new dynamic Web project in Eclipse:
- On the main menu select File > New > Project....
- In the upcoming wizard choose Web > Dynamic Web Project.



- Click **Next**.
- Enter project name as "jsp-servlet-jdbc-mysql-example
- Make sure that the target runtime is set to Apache Tomcat with the currently supported version.

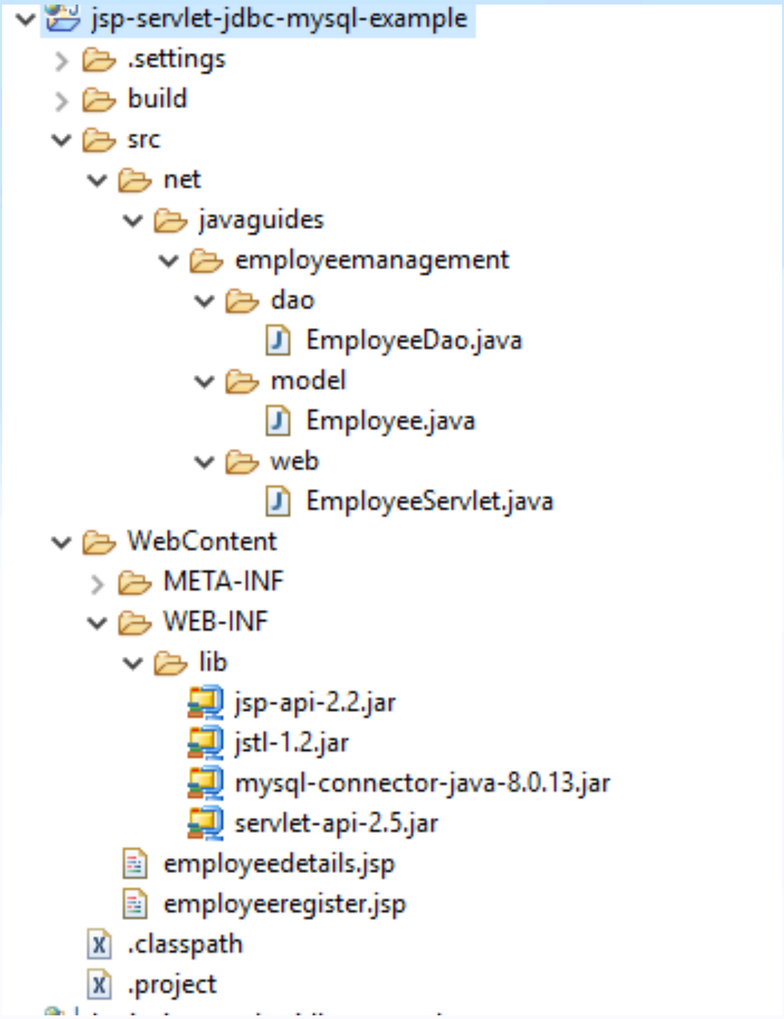
Add Dependencies

Add the latest release of below jar files to the ***lib*** folder.

- jsp-api.2.3.1.jar
- servlet-api.2.3.jar
- mysql-connector-java-8.0.13.jar
- jstl-1.2.jar

Project Structure

- Standard project structure for your reference -



MySQL Database Setup

Let's create a database named "employees" in MySQL. Now, let's create an employee table using below DDL script:

```
CREATE TABLE `employee` (
  `id` int(3) NOT NULL AUTO_INCREMENT,
  `first_name` varchar(20) DEFAULT NULL,
  `last_name` varchar(20) DEFAULT NULL,
  `username` varchar(250) DEFAULT NULL,
  `password` varchar(20) DEFAULT NULL,
  `address` varchar(45) DEFAULT NULL,
  `contact` varchar(45) DEFAULT NULL,
  PRIMARY KEY (`id`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4
 COLLATE=utf8mb4_0900_ai_ci;
```

Create a JavaBean - Employee.java

Let's create an *Employee* JavaBean class which we will use in JSP action tags.

```
package net.javaguides.jsp.jdbc.bean;
import java.io.Serializable;
public class Employee implements Serializable {
private static final long serialVersionUID = 1 L;
    private String firstName;
    private String lastName;
    private String username;
    private String password;
    private String address;
    private String contact;
    public String getFirstName() {
        return firstName;
    }
```

Create an EmployeeDao.java

- Let's create EmployeeDao class which contains JDBC code to connect with MySQL database. Add the following code to n EmployeeDao class:

```
package net.javaguides.jsp.jdbc.database;
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.PreparedStatement;
import java.sql.SQLException;
import net.javaguides.jsp.jdbc.bean.Employee;
```

Create an EmployeeDao.java

```
public class EmployeeDao {
  public int registerEmployee(Employee employee) throws
ClassNotFoundException {
      String INSERT_USERS_SQL = "INSERT INTO employee" +
       "  (id, first_name, last_name, username, password, address, contact) VALUES
" +
          " (?, ?, ?, ?, ?,?,?);";
 int result = 0;
     Class.forName("com.mysql.jdbc.Driver");
 try (Connection connection = DriverManager
 .getConnection("jdbc:mysql://localhost:3306/employees?useSSL=false", "root",
"root");
```

```
PreparedStatement preparedStatement =
connection.prepareStatement(INSERT_USERS_SQL)) {
        preparedStatement.setInt(1, 1);
        preparedStatement.setString(2, employee.getFirstName());
        preparedStatement.setString(3, employee.getLastName());
        preparedStatement.setString(4, employee.getUsername());
        preparedStatement.setString(5, employee.getPassword());
        preparedStatement.setString(6, employee.getAddress());
        preparedStatement.setString(7, employee.getContact());

        System.out.println(preparedStatement);
```

```
result = preparedStatement.executeUpdate();

    } catch (SQLException e) {
        // process sql exception
        printSQLException(e);
    }
    return result;
}

private void printSQLException(SQLException ex) {
    for (Throwable e: ex) {
        if (e instanceof SQLException) {
            e.printStackTrace(System.err);
```

```
System.err.println("SQLState: " + ((SQLException) e).getSQLState());
            System.err.println("Error Code: " + ((SQLException) e).getErrorCode());
            System.err.println("Message: " + e.getMessage());
            Throwable t = ex.getCause();
            while (t != null) {
                System.out.println("Cause: " + t);
                t = t.getCause();
            }
        }
    }
  }
}
```

Create an EmployeeServlet.java

Let's create an EmployeeServlet class to process HTTP request parameters and redirect to the appropriate JSP page after request data stored in the database:

```
package net.javaguides.employeemanagement.web;

import java.io.IOException;

import javax.servlet.ServletException;
import javax.servlet.annotation.WebServlet;
import javax.servlet.http.HttpServlet;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;

import net.javaguides.employeemanagement.dao.EmployeeDao;
import net.javaguides.employeemanagement.model.Employee;
```

Create an EmployeeServlet.java

```
@WebServlet("/register")
public class EmployeeServlet extends HttpServlet {
    private static final long serialVersionUID = 1 L;
    private EmployeeDao employeeDao;

    public void init() {
        employeeDao = new EmployeeDao();
    }

    protected void doPost(HttpServletRequest request, HttpServletResponse
response)
    throws ServletException, IOException {
```

Create an EmployeeServlet.java

```
String firstName = request.getParameter("firstName");
    String lastName = request.getParameter("lastName");
    String username = request.getParameter("username");
    String password = request.getParameter("password");
    String address = request.getParameter("address");
    String contact = request.getParameter("contact");
```

Create an EmployeeServlet.java

```
Employee employee = new Employee();
    employee.setFirstName(firstName);
    employee.setLastName(lastName);
    employee.setUsername(username);
    employee.setPassword(password);
    employee.setContact(contact);
    employee.setAddress(address);
```

Create an EmployeeServlet.java

```
try {
        employeeDao.registerEmployee(employee);
    } catch (Exception e) {
        // TODO Auto-generated catch block
        e.printStackTrace();
    }

    response.sendRedirect("employeedetails.jsp");
  }
```

Create a employeeregister.jsp

Let's design employee registration HTML form with the following fields:

- firstName
- lastName
- username
- password
- address
- contact

Create a employeeregister.jsp

```
<%@ page language="java" contentType="text/html; charset=ISO-8859-1"
 pageEncoding="ISO-8859-1"%>
<!DOCTYPE html>
<html>
<head>
<meta charset="ISO-8859-1">
<title>Insert title here</title>
</head>
<body>
 <div align="center">
  <h1>Employee Register Form</h1>
  <form action="<%= request.getContextPath() %>/register" method="post">
   <table style="with: 80%">
    <tr>
```

Create a employeeregister.jsp

```
<td>First Name</td>
  <td><input type="text" name="firstName" /></td>
 </tr>
 <tr>
  <td>Last Name</td>
  <td><input type="text" name="lastName" /></td>
 </tr>
 <tr>
  <td>UserName</td>
  <td><input type="text" name="username" /></td>
 </tr>
 <tr>
  <td>Password</td>
  <td><input type="password" name="password" /></td>
 </tr>
 <tr>
```

Create a employeeregister.jsp

```
 <td>Address</td>
   <td><input type="text" name="address" /></td>
  </tr>
  <tr>
   <td>Contact No</td>
   <td><input type="text" name="contact" /></td>
  </tr>
 </table>
 <input type="submit" value="Submit" />
</form>
</div>
</body>
</html>
```

Create an employeedetails.jsp

- After an employee successfully registered then this page show a successful message on screen:

```
<%@ page language="java" contentType="text/html; charset=ISO-8859-1"
 pageEncoding="ISO-8859-1"%>
<%@page import="net.javaguides.employeemanagement.dao.*"%>
<!DOCTYPE html>
<html>
<head>
<meta charset="ISO-8859-1">
<title>Insert title here</title>
</head>
```

Create an employeedetails.jsp

```
<body>
 <h1>User successfully registered!</h1>
</body>
</html>
```

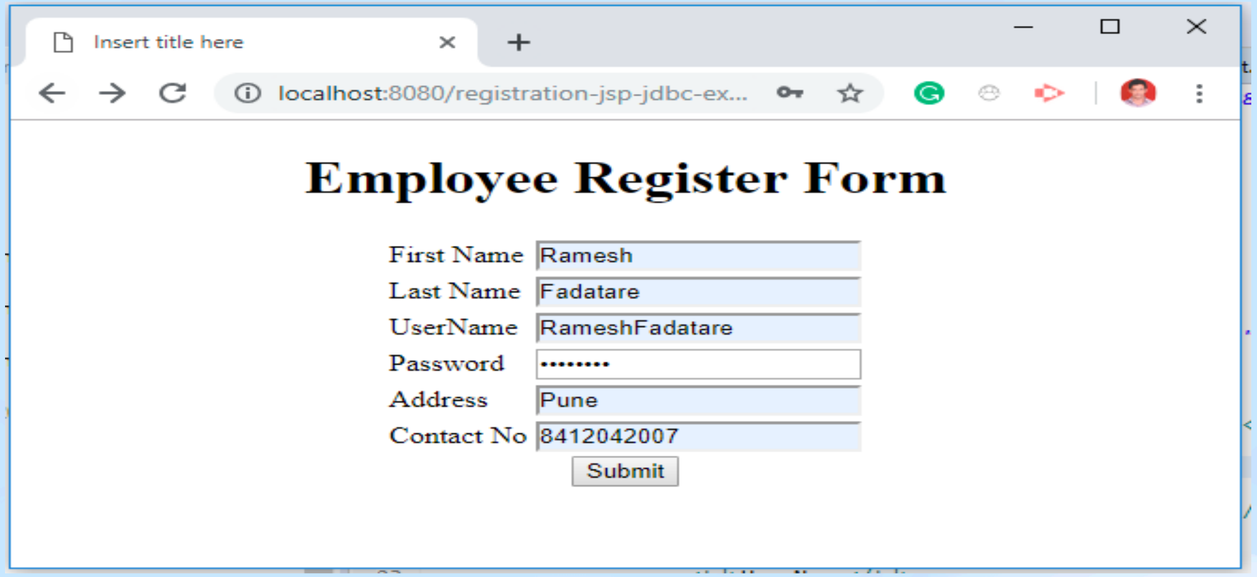
- Note that in the above page, we have used JSP action tags. Read more about action tags here

Demo

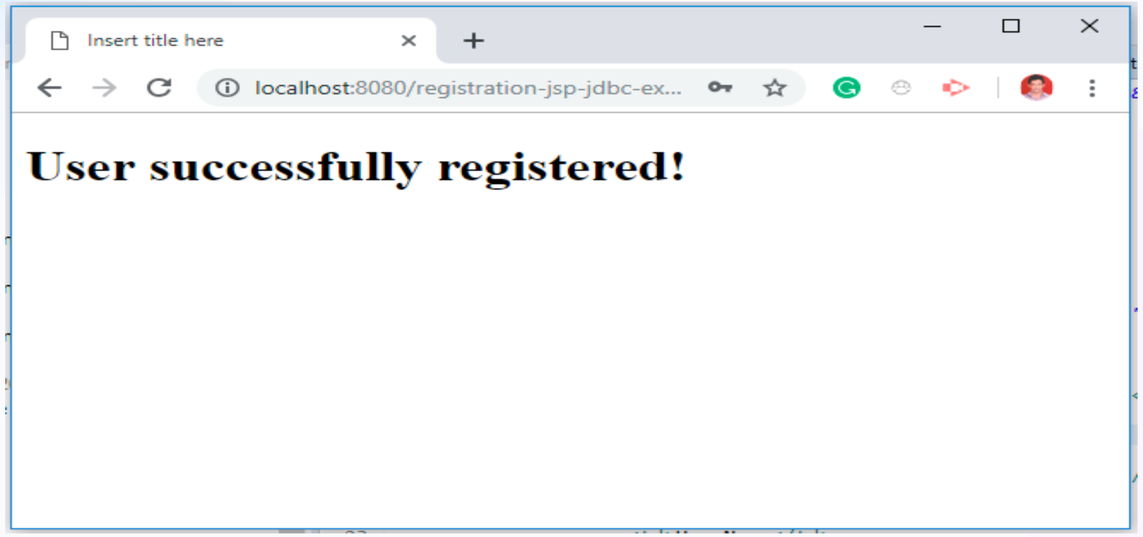
- It's time to see a demo of the above development.
- Deploy this web application in tomcat server.

Employee Registration

- Once you deploy this application successfully



Registration Success Page :



Introduction to Servlet Session Management

- Servlet Session Management is a mechanism in Java used by Web container to store session information.

- Session tracking is a way to manage the data of a user, this is known as session management in servlet.

- Session in Java are managed through different ways, such as, HTTP Session API, Cookies, URL rewriting, etc.

Session Management/ Tracking Methods

User Authorization: It is one way where user provides Username and password or any authentication credentials from login and then these are passed via server and client to maintain the servlet session.

URL rewriting: User can append session identifier parameter with every request and response to keep track of session.

Hidden Fields: User has access to create unique field in HTML which is hidden, when user starts navigating, user will be able to set the value uniquely to customer and have track over the session

Session Management/ Tracking Methods

Session Tracking API: It is built on top of all other Tracking methods. This type of session tracking is used for developers to minimize overhead of session tracking. Major disadvantage is that most of the time, user need not have to track session, but need to store some data in the session that can be used in future requests.

Cookies: Cookie is a key value pair of information sent by server to browsers. It is the most used technology for session tracking. Cookie is a smallest piece of information sent by the web server in head tag and is stored as browser cookie.

Two types in Cookies:

Non-persistent Cookie: It is valid only for single session and is removed each time when browser gets closed.

Persistent Cookie: It is valid for multiple sessions and is removed only when user logs out but not when browsers get closed.

HTTP and SSL: Browsers that support Secure Socket Layer communication use SSL support via HTTPS to generate unique session key as part of encrypted conversation. Modern sites like e-commerce, ticket booking, Internet banking, etc., use HTTPs to securely transfer data and manage session.

Key Method Used in HTTPSession

isNew(): Returns true is user does not know about the session. If cookies are disabled, then session is new.
getId(): Returns string that contains unique identifier that is assigned to this session. Is used while using URL rewriting the session.
getAttribute(): Returns the object bound in present session.
setAttribute(): Binds object to present session, uses specified name.
invalidate(): Expires current session and unbinds the object binded
setMaxInactiveInterval(): Specifies time between client requests before servlet invalidates session. Negative time indicates session shouldn't timeout.

Adding JSTL support

JSTL

- JSTL stands for JSP Standard Tag Library. JSTL is the standard tag library that provides tags to control the JSP page behavior.

- JSTL tags can be used for iteration and control statements, internationalization, SQL etc.

- JSP code like HTML but their functionality is very limited. For example, we can’t loop through a collection using EL or action elements and we can’t escape HTML tags to show them like text in client side.

- This is where JSTL tags comes handy because we can do much more from JSTL tags.

- JSTL is part of the Java EE API and included in most servlet containers.
- Most of the times, you can find them in the example projects of server download and you can use them

JSTL jars :

- JSTL jars are container specific, for example in Tomcat, we need to include jstl.jar and standard.jar jar files in project build path.

- If you have maven project, below dependencies should be added in pom.xml file or else you will get following error in JSP pages

```
<dependency>
        <groupId>jstl</groupId>
        <artifactId>jstl</artifactId>
        <version>1.2</version>
</dependency>
<dependency>
        <groupId>taglibs</groupId>
        <artifactId>standard</artifactId>
        <version>1.1.2</version>
</dependency>
```

JSTL Tags :

Based on the JSTL functions, they are categorized into five types.

JSTL Core Tags: JSTL Core tags provide support for iteration, conditional logic, catch exception, url, forward or redirect response etc.

<%@ taglib uri="https://java.sun.com/jsp/jstl/core" prefix="c" %>

JSTL Formatting and Localisation Tags: JSTL Formatting tags are provided for formatting of Numbers, Dates and i18n support through locales and resource bundles.

<%@ taglib uri="https://java.sun.com/jsp/jstl/fmt" prefix="fmt" %>

JSTL Tags :

JSTL SQL Tags: JSTL SQL Tags provide support for interaction with relational databases such as Oracle, MySql etc. Using JSTL SQL tags we can run database queries

```
<%@ taglib uri="https://java.sun.com/jsp/jstl/sql" prefix="sql" %>
```

JSTL XML Tags: JSTL XML tags are used to work with XML documents such as parsing XML, transforming XML data and XPath expressions evaluation.

```
<%@ taglib uri="https://java.sun.com/jsp/jstl/xml" prefix="x" %>
```

JSTL Tags :

JSTL Functions Tags: JSTL tags provide a number of functions that we can use to perform common operation, most of them are for String manipulation such as String Concatenation, Split String etc.

<%@ taglib uri="https://java.sun.com/jsp/jstl/functions" prefix="fn" %>

JSTL Core Tags

JSTL Core Tag	Description
<c:out>	To write something in JSP page, we can use EL also with this tag
<c:import>	Same as jsp:include or include directive
<c:redirect>	redirect request to another resource

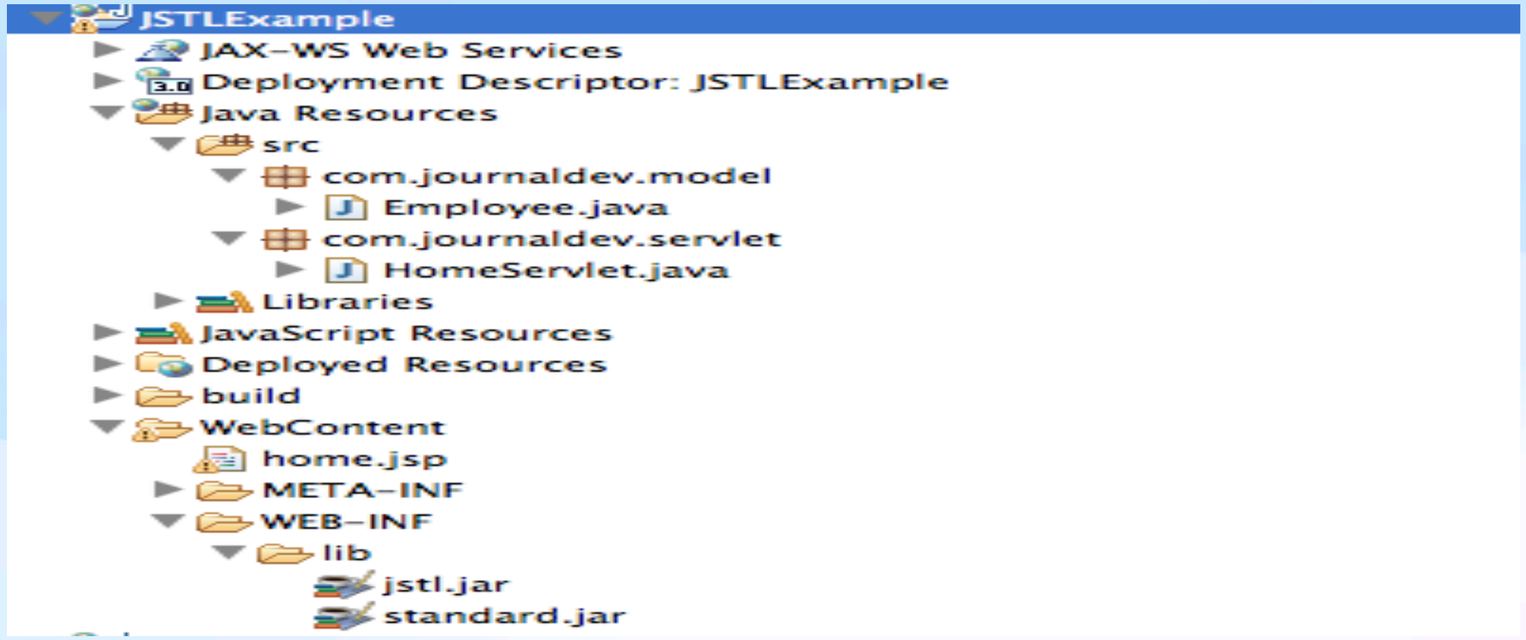
JSTL Core Tags

<c:if>	Simple conditional logic, used with EL and we can use it to process the exception from <c:catch>
<c:choose>	Simple conditional tag that establishes a context for mutually exclusiv conditional operations, marked by <c:when> and <c:otherwise>
<c:when>	Subtag of <c:choose> that includes its body if its condition evalutes to 'true'
<c:set>	To set the variable value in given scope.
<c:remove>	To remove the variable from given scope
<c:catch>	To catch the exception and wrap it into an object.

JSTL Core Tags

<c:otherwise>	Subtag of <c:choose> that includes its body if its condition evalutes to ‘false’.
<c:forEach>	for iteration over a collection
<c:forTokens>	for iteration over tokens separated by a delimiter.
<c:param>	used with <c:import> to pass parameters

JSTL Tutorial :



JSTL Tutorial - Java Bean Class :

```
package com.journaldev.model;
public class Employee {
        private int id;
        private String name;
        private String role;
        public Employee() {
        }
         public int getId() {
                return id;
```

JSTL Tutorial - Java Bean Class :

```
}
        public void setId(int id) {
                this.id = id;
        }
        public String getName() {
                return name;
        }
        public void setName(String name) {
                this.name = name;
        }
```

JSTL Tutorial - Java Bean Class :

```
        public String getRole() {
                return role;
        }
        public void setRole(String role) {
                this.role = role;
        }
}
```

Uploading Files with Servlets and JSP

- First, let's add a form to our HTML file:

```
<form method="post" action="multiPartServlet" enctype="multipart/form-data">
   Choose a file: <input type="file" name="multiPartServlet" />
  <input type="submit" value="Upload" />
</form>
```

```
@MultipartConfig(fileSizeThreshold = 1024 * 1024,
  maxFileSize = 1024 * 1024 * 5,
  maxRequestSize = 1024 * 1024 * 5 * 5)
  public class MultipartServlet extends HttpServlet {
}
```

- Then, let's make sure that our default server upload folder is set:

```
String uploadPath = getServletContext().getRealPath("") +
File.separator + UPLOAD_DIRECTORY;
File uploadDir = new File(uploadPath);
if (!uploadDir.exists()) uploadDir.mkdir();
```

- Finally, we can easily retrieve our inbound *File* from the *request* using the *getParts()* method, and save it to the disk:

```
for (Part part : request.getParts()) {
    fileName = getFileName(part);
    part.write(uploadPath + File.separator + fileName);
}
```

- Note that, in this example, we're using a helper method getFileName():

```
private String getFileName(Part part) {
    for (String content : part.getHeader("content-disposition").split(";")) {
        if (content.trim().startsWith("filename"))
            return content.substring(content.indexOf("=") + 2, content.length() -1);
        }
    return Constants.DEFAULT_FILENAME;
}
```

For Servlet 3.1. projects, we could alternatively use the Part.getSubmittedFileName() method:

```
fileName = part.getSubmittedFileName();
```

Setup:

```
<dependency>
   <groupId>commons-fileupload</groupId>
   <artifactId>commons-fileupload</artifactId>
   <version>1.4</version>
</dependency>
```

Setup:

```
<dependency>
   <groupId>commons-io</groupId>
   <artifactId>commons-io</artifactId>
   <version>2.11.0</version>
</dependency>
```

Upload Servlet

The three main parts to incorporating Apache's *FileUpload* library go as follows:

- An upload form in a *.jsp* page.
- Configuring your *DiskFileItemFactory* and *ServletFileUpload* object.
- Processing the actual contents of a multipart file upload.

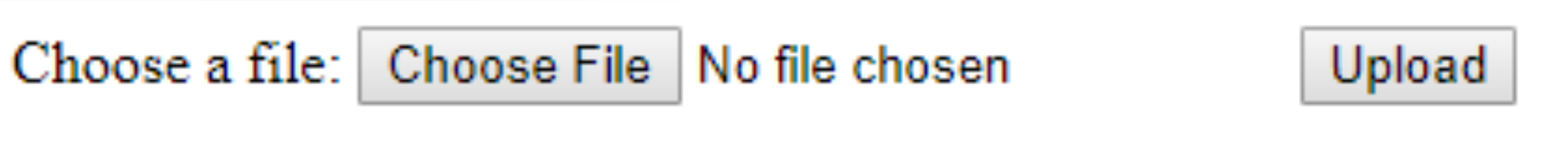
- The upload form is the same as the one in the previous section.
- Let's move on to creating our Jakarta EE servlet.
- In our request processing method, we can wrap the incoming HttpRequest with a check to see if it's a multi-part upload.

```
if (ServletFileUpload.isMultipartContent(request)) {
    DiskFileItemFactory factory = new DiskFileItemFactory();
    factory.setSizeThreshold(MEMORY_THRESHOLD);
    factory.setRepository(new File(System.getProperty("java.io.tmpdir")));
```

```
ServletFileUpload upload = new ServletFileUpload(factory);
   upload.setFileSizeMax(MAX_FILE_SIZE);
   upload.setSizeMax(MAX_REQUEST_SIZE);
  String uploadPath = getServletContext().getRealPath("")
+ File.separator + UPLOAD_DIRECTORY;
   File uploadDir = new File(uploadPath);
   if (!uploadDir.exists()) {
      uploadDir.mkdir();
   }
}
```

Running the Example

- After we've compiled our project into a *.war*, we can drop it into our local Tomcat instance and start it up.
- From there, we can bring up the main upload view where we're presented with a form:
- After successfully uploading our file, we should see the message



File photo-1504466664756-1adbe6d13b36.jpg has uploaded successfully!

Running the Example

- After we've compiled our project into a *.war*, we can drop it into our local Tomcat instance and start it up.

- From there, we can bring up the main upload view where we're presented with a form:

- After successfully uploading our file, we should see the message